

BỘ NÔNG NGHIỆP VÀ MÔI TRƯỜNG
PHÂN HIỆU TRƯỜNG ĐẠI HỌC THỦY LỢI
BỘ MÔN CÔNG NGHỆ THÔNG TIN



BÁO CÁO MÔN KHAI PHÁ DỮ LIỆU

KHÓA HỌC: S25-64CNTT

Tên đề tài: Xây dựng mô hình dự đoán ảnh phổ

Giảng viên hướng dẫn: Th.S Vũ Thị Hạnh

Sinh viên thực hiện: Nguyễn Thái Anh Minh

MSSV: 2251068210

Thái Quang Minh

MSSV:2251068212

Hoàng Đức Công Tú

MSSV: 2251068274

TP.HCM, tháng 10 năm 2025

TÓM TẮT

Trong bối cảnh các bệnh lý về đường hô hấp đặc biệt là viêm phổi do vi khuẩn và virus đang ngày càng phổ biến ở trẻ nhỏ, việc đưa ứng dụng trí tuệ nhân tạo vào chẩn đoán hình ảnh y khoa đang dần trở nên cực kỳ cần thiết và hữu ích thì đề tài “Xây dựng mô hình dự đoán ảnh phổi” ra đời là một điều tất yếu, hướng đến việc phát triển hệ thống hỗ trợ chẩn đoán y khoa có khả năng phân tích và dự đoán tình trạng phổi của trẻ em thông qua hình ảnh X-quang, nhằm hỗ trợ các y bác sĩ trong quá trình chẩn đoán bệnh.

Mục tiêu của đề án “Xây dựng mô hình dự đoán ảnh phổi” là xây dựng và phát triển một hệ thống sao cho hệ thống có thể phân loại chính xác được bệnh nhi nào không mắc bệnh và bị viêm phổi (PNEUMONIA) do vi khuẩn và virus. Trong quá trình nghiên cứu, nhóm đã tiến hành thử nghiệm trên nhiều mô hình học sâu khác nhau như CNN, DenseNet, MobileNetV2, XceptionNet,... để so sánh hiệu suất của từng mô hình từ đó chọn ra được một mô hình tốt nhất và cải tiến mô hình đó sao cho ra được kết quả tốt nhất. Không chỉ dừng lại ở việc lấy mô hình nào với kết quả tốt nhất mà còn phải tiến hành phân tích các đặc trưng của ảnh phổi vì tính phức tạp của ảnh phổi X-quang và cũng nhằm hiểu rõ hơn cơ chế hoạt động và khả năng nhận diện của mô hình. Kết quả mà nhóm mong muốn đạt được sau khi xây dựng và phát triển mô hình học trên chính là khả năng dự đoán chính xác và ổn định của mô hình, góp phần hỗ trợ quá trình chẩn đoán hình ảnh y khoa trong thực tế.

Bài toán cần phải giải quyết từ đề án trên đó là cho một ảnh X-quang phổi (có thể là bệnh hoặc không bệnh) và xây dựng mô hình sao cho nó có thể phân loại đúng tình trạng phổi (do vi khuẩn hay virus). Giải thích kết quả dự đoán thông qua kỹ thuật Grad-CAM, thu thập thông tin phổi bệnh hay không bệnh từ các nguồn uy tín. Sau cùng là đánh giá và so sánh hiệu suất giữa các mô hình để tìm ra mô hình nào tốt nhất trong việc phân loại ảnh, điều cực kỳ quan trọng trong bộ dữ liệu y khoa trên.

MỤC LỤC

CHƯƠNG 1: GIỚI THIỆU VỀ BỘ DỮ LIỆU	1
1.1 Nguồn của bộ dữ liệu	1
1.2 Kích thước bộ dữ liệu	1
1.3 Mô tả bộ dữ liệu	1
CHƯƠNG 2: CƠ SỞ LÝ THUYẾT	2
2.1 Cách đọc ảnh X-quang phổi:	2
2.2 Phân biệt phổi bình thường và phổi bị viêm do bacteria (vi khuẩn) hay virus	6
2.2.1 Phổi bình thường	6
2.2.2 Viêm phổi do bacteria (vi khuẩn)	7
2.2.3 Viêm phổi do Virus	8
CHƯƠNG 3: TIỀN XỬ LÝ DỮ LIỆU VÀ XÂY DỰNG MÔ HÌNH	10
3.1 Sử dụng ImageDataGenerator để tiền xử lý	10
3.2 Đọc và chuẩn bị hình ảnh	11
3.3 Xây dựng mô hình:	13
3.4 Kiểm tra kết quả Train mô hình	16
3.5 Fine tuning	19
3.6 Kết quả mô hình trên web	21
CHƯƠNG 4: PHÂN TÍCH ẢNH & KẾT QUẢ	24
4.1 Grad-CAM	24
4.2 Code thực hiện và giải thích	24
4.3 Kết quả và phân tích với ảnh gốc	31
4.3.1 Ảnh 1	31
4.3.2 Ảnh 2	32
4.3.3 Ảnh 3	33

CHƯƠNG 5: KẾT LUẬN	35
5.1 Tổng kết nội dung đã thực hiện	35
5.2 Khó khăn, thách thức và cách giải quyết	35
HƯỚNG PHÁT TRIỂN	37
TÀI LIỆU THAM KHẢO	38

MỤC LỤC HÌNH ẢNH VÀ BẢNG

Bảng 1: Bảng quy ước độ đậm của ảnh X-quang	2
Bảng 2: Nguyên tắc đọc X-quang theo thứ tự	3
Bảng 3: Bảng giải thích thuật ngữ theo hình 2	6
Hình 1: Ảnh X-Quang tiêu chuẩn	2
Hình 2: Các thuật ngữ mô tả vị trí cơ quan phổi của ảnh X-quang	4
Hình 3: X-quang phổi bình thường.....	7
Hình 4: X-quang phổi do vi khuẩn	8
Hình 5: X-quang phổi do virus	9
Hình 6: Xử lý, tăng cường dữ liệu ảnh	10
Hình 7: Đọc và chuẩn bị hình ảnh	11
Hình 8: Tạo generator cho train, validation và test.....	12
Hình 9: Xây dựng mô hình XceptionNet.....	13
Hình 10: Đóng băng trọng số base model.....	14
Hình 11: Tạo mô hình mới bằng cách xếp chồng các lớp.....	14
Hình 12: Compile mô hình	15
Hình 13: Earlystop và Callback nếu val_loss không giảm	15
Hình 14: Huấn luyện mô hình.....	16
Hình 15: Kết quả huấn luyện mô hình.....	16
Hình 16: Độ chính xác và mất mát qua từng epoch.....	17
Hình 17: Ma trận nhầm lẫn giữa NORMAL và PNEUMONIA	18
Hình 18: Báo cáo phân loại.....	18
Hình 19: FineTuning mô hình.....	19
Hình 20: Kết quả huấn luyện sau khi FineTuning	20
Hình 21: Ma trận nhầm lẫn sau khi FineTuning	20
Hình 22: Báo cáo phân loại sau khi FineTuning.....	21
Hình 23: Ảnh dự đoán và so sánh mô hình trên web 1	22
Hình 24: Ảnh dự đoán và so sánh mô hình trên web 2	22
Hình 25: Ảnh dự đoán và so sánh mô hình trên web 3	23
Hình 26: Ảnh dự đoán và so sánh mô hình trên web 4.....	23

Hình 27: Tải lại mô hình Xception để tạo Grad-CAM	24
Hình 28: Hàm tạo Grad-CAM	24
Hình 29: Hàm xây model trung gian để lấy input và output.....	25
Hình 30: Chạy forward qua Xception.....	26
Hình 31: Tính gradient.....	26
Hình 32: Tạo bản đồ nhiệt	26
Hình 33: Code trả về heatmap	27
Hình 34: Xử lý ảnh đầu vào.....	27
Hình 35: Dự đoán mô hình	28
Hình 36: Chuẩn bị tạo Grad-CAM.....	28
Hình 37: Kết quả.....	29
Hình 38: Resize heatmap	29
Hình 39: Chồng ảnh.....	30
Hình 40: In ra ảnh gốc, heatmap và ảnh chồng hiển thị vùng mô hình dự đoán	30
Hình 41: Ảnh sau khi mô hình tạo Grad-CAM 1.....	31
Hình 42: Ảnh gốc đã phân tích 1	31
Hình 43: Ảnh sau khi mô hình tạo Grad-CAM 2.....	32
Hình 44: Ảnh gốc đã phân tích 2	32
Hình 45: Ảnh sau khi mô hình tạo Grad-CAM 3.....	33
Hình 46: Ảnh gốc đã phân tích 3	34

CHƯƠNG 1: GIỚI THIỆU VỀ BỘ DỮ LIỆU

1.1 Nguồn của bộ dữ liệu

- Nguồn của bộ dữ liệu được lấy trên trang web kaggle.com.
- Link : <https://www.kaggle.com/datasets/ghost5612/chest-x-ray-images-normal-and-pneumonia>

1.2 Kích thước bộ dữ liệu

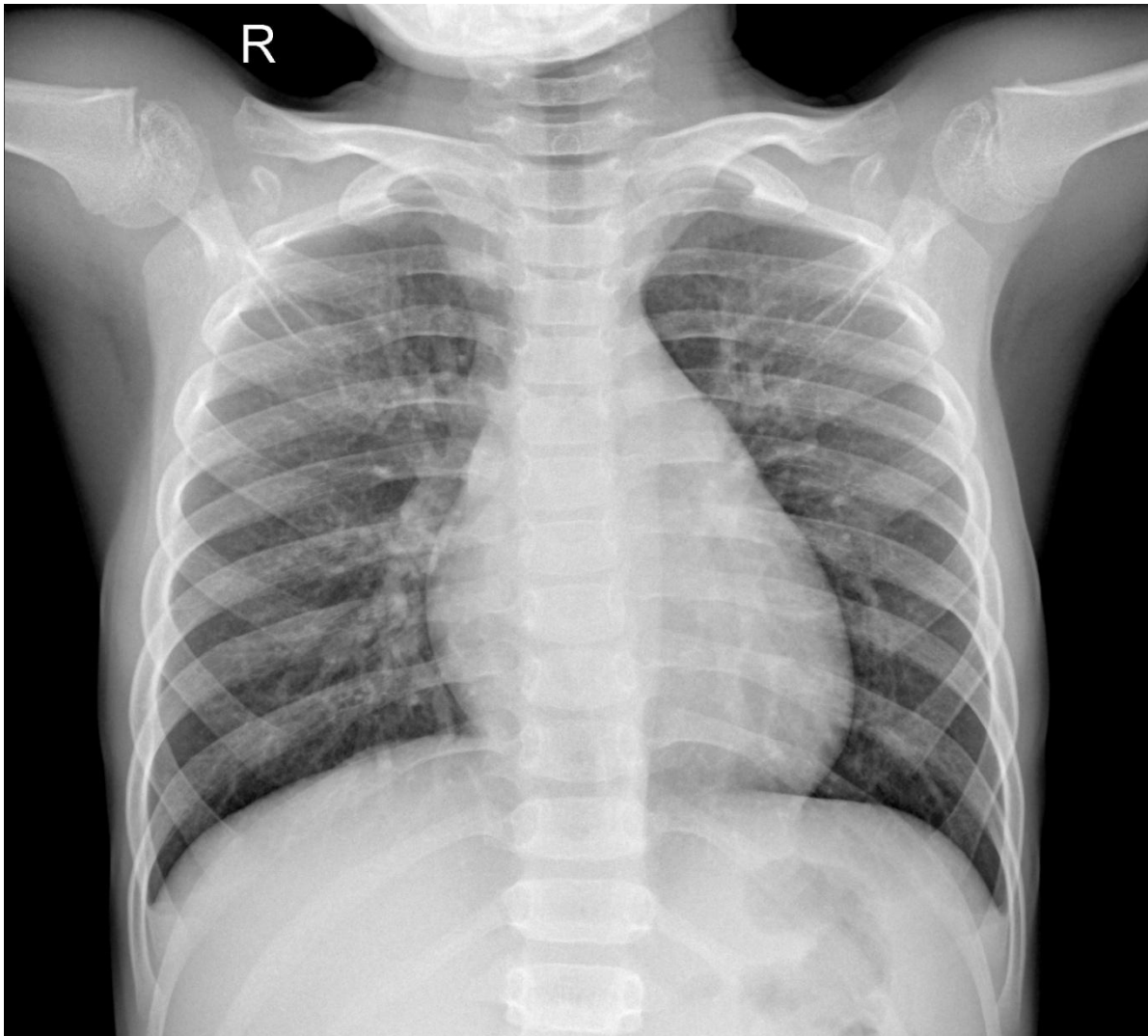
- Bộ dữ liệu gồm có 5887 ảnh dạng jpeg gồm 3 thư mục chứa ảnh: train, val, test.
- Trong mỗi thư mục train, val, test có thêm 2 thư mục là NORMAL (phổi bình thường) và PNEUNOMIA (phổi bị viêm)
- Trong thư mục train có 5216 ảnh trong đó NORMAL có 1341 ảnh (25,7%) và PNEUNOMIA có 3875 ảnh (74,3%).
- Trong thư mục val có 47 ảnh trong đó NORMAL có 24 ảnh (51%) và PNEUNOMIA có 23 ảnh (49%).
- Trong thư mục test có 624 ảnh trong đó NORMAL có 234 ảnh (37.5%) và PNEUNOMIA có 390 ảnh (62.5%).

1.3 Mô tả bộ dữ liệu

- Hình ảnh X-Quang ngực được tổng hợp từ một nhóm bệnh nhi từ 1 đến 5 tuổi tại Trung tâm Y tế Phụ nữ và Trẻ em Quảng Châu, Quảng Châu và được phân loại bởi hai chuyên gia trước khi được đưa vào đào tạo cho hệ thống AI.

CHƯƠNG 2: CƠ SỞ LÝ THUYẾT

2.1 Cách đọc ảnh X-quang phổi:



Hình 1: Ảnh X-Quang tiêu chuẩn

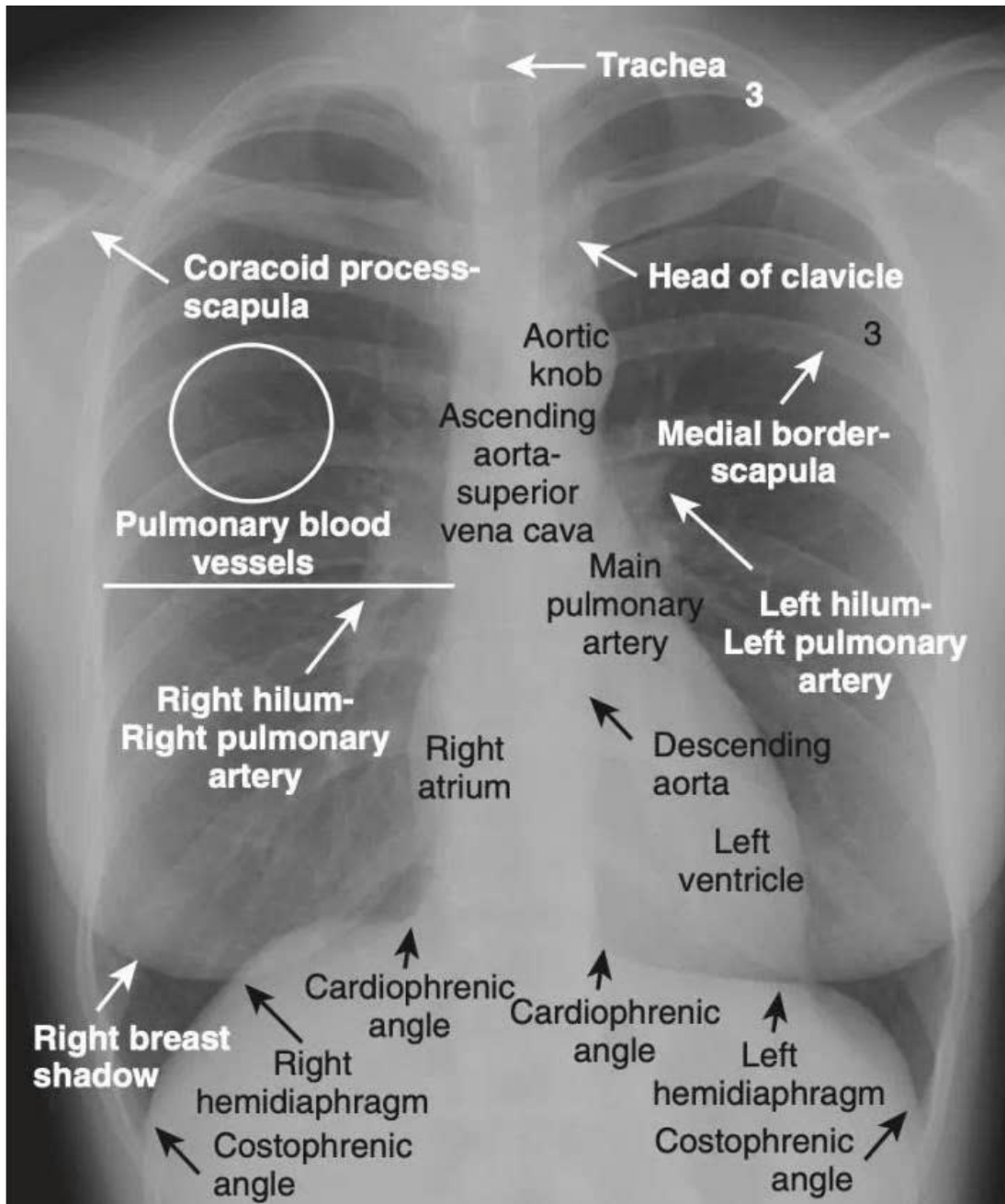
Năm độ đậm của X-quang quy ước	
Khí (Air)	Màu đen (Black)
Mô mỡ (fat)	Xám đậm (Dark grey image)
Mô mềm (soft tissue)	Xám (Grey)
Xương hoặc Vôi (bone, calcium)	Trắng vừa (off-white)
Kim loại (Metal)	Rất trắng (Bright white)

Bảng 1: Bảng quy ước độ đậm của ảnh X-quang

- Từ thời trước nguyên tắc đọc X-quang từ ngoài vào trong, từ trong ra ngoài. Nhưng gần đây thì có cách đọc X-quang kiểu mới là theo thứ tự A – B – C – D – E.

Ký hiệu	Ý nghĩa/Mục đích	Nội dung quan sát
A – Airway (Đường thở)	Kiểm tra đường thở có thông suốt, không bị lệch hoặc tắc nghẽn.	Quan sát khí quản có nằm giữa không, có bị ép lệch do u, tràn khí, hoặc xẹp phổi không
B – Breathing & Pleura (Hô hấp & màng phổi)	Đánh giá phổi và màng phổi.	Xem có vùng mờ (viêm phổi), sáng bất thường (tràn khí), hay mức dịch (tràn dịch màng phổi) không.
C – Cardiac & Circulation (Tim mạch & tuần hoàn)	Đánh giá hình dạng và kích thước tim, cũng như mạch máu lớn.	Xem tim có to không (bình thường < 50% chiều ngang ngực), mạch máu phổi có giãn hay ứ huyết không.
D – Diaphragm (Cơ hoành)	Quan sát vị trí và hình dạng cơ hoành.	Hai vòm hoành có cân xứng không, có khí bất thường dưới hoành (nghi thủng tạng rỗng) không.
E – Everything else (Phần còn lại)	Kiểm tra toàn bộ các cấu trúc khác.	Xem xương sườn, cột sống, mô mềm, và vùng phổi còn lại để phát hiện bất thường khác.

Bảng 2: Nguyên tắc đọc X-quang theo thứ tự



Hình 2: Các thuật ngữ mô tả vị trí cơ quan phổi của ảnh X-quang

Thuật ngữ tiếng Anh	Nghĩa tiếng Việt	Giải thích
Trachea	Khí quản	Ống dẫn không khí từ họng xuống phổi

Coracoid process - scapula	Mỏm quạ – xương vai	Phần xương vai (xương bả) nhìn thấy ở phía trên phổi
Head of clavicle	Đầu xương đòn	Nối giữa xương đòn và xương ức
Medial border - scapula	Bờ trong của xương vai	Cạnh trong của xương bả vai
Aortic knob	Cung động mạch chủ	Chỗ cong của động mạch chủ, thấy rõ bên trái tim
Ascending aorta – superior vena cava	Động mạch chủ lên – tĩnh mạch chủ trên	Mạch máu lớn đi từ tim ra (động mạch) và trở về tim (tĩnh mạch)
Main pulmonary artery	Động mạch phổi chính	Dẫn máu từ tim phải lên phổi
Right hilum – Right pulmonary artery	Rốn phổi phải – Động mạch phổi phải	Vùng trung tâm phổi nơi mạch máu và phế quản đi vào/ra
Left hilum – Left pulmonary artery	Rốn phổi trái – Động mạch phổi trái	Tương tự bên phải
Pulmonary blood vessels	Mạch máu phổi	Các nhánh mạch nhỏ tỏa trong phổi, nhìn thấy như các vết xám
Right atrium	Tâm nhĩ phải	Buồng tim nhận máu tĩnh mạch từ cơ thể
Left ventricle	Tâm thất trái	Buồng tim bơm máu ra toàn thân
Descending aorta	Động mạch chủ xuống	Đưa máu từ tim xuống phần thân dưới
Right breast shadow	Bóng tuyến vú phải	Vùng mờ do mô vú che phủ (thường thấy ở nữ)
Right hemidiaphragm	Cơ hoành phải	Vòm cơ phân cách ngực và bụng, thường cao hơn bên trái

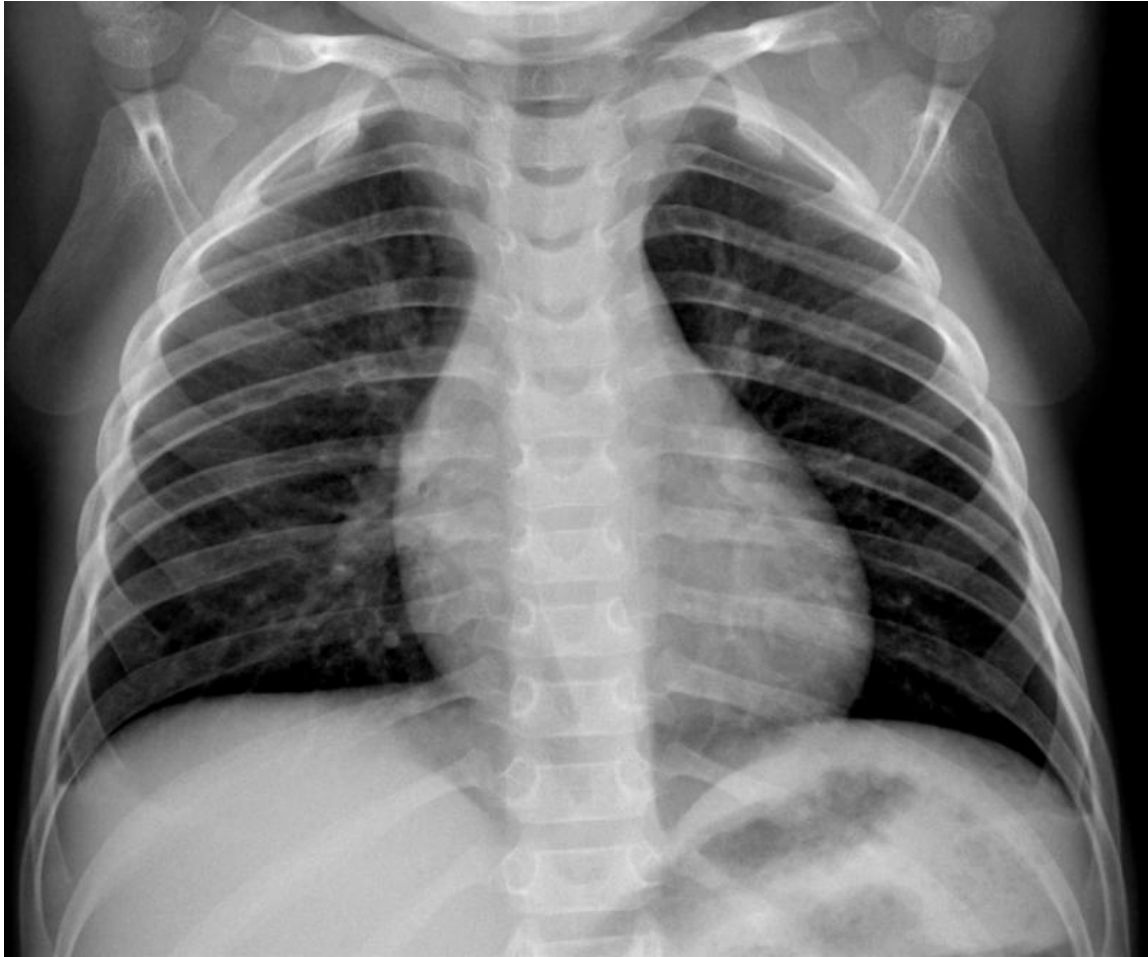
Left hemidiaphragm	Cơ hoành trái	Vòm cơ bên trái, thấp hơn do có tim đè lên
Cardiophrenic angle	Góc tim – hoành	Góc giữa tim và cơ hoành (cần rõ ràng trên ảnh bình thường)
Costophrenic angle	Góc sườn – hoành	Góc giữa xương sườn và cơ hoành – nếu mờ có thể là tràn dịch màng phổi
Right hilum – Right pulmonary artery	Rốn phổi phải – Động mạch phổi phải	Trung tâm mạch máu phổi bên phải
3 (cạnh phổi phải và trái)	Xương sườn thứ 3	Dấu mốc định vị giải phẫu trên ảnh

Bảng 3: Bảng giải thích thuật ngữ theo hình 2

2.2 Phân biệt phổi bình thường và phổi bị viêm do bacteria (vi khuẩn) hay virus

2.2.1 Phổi bình thường

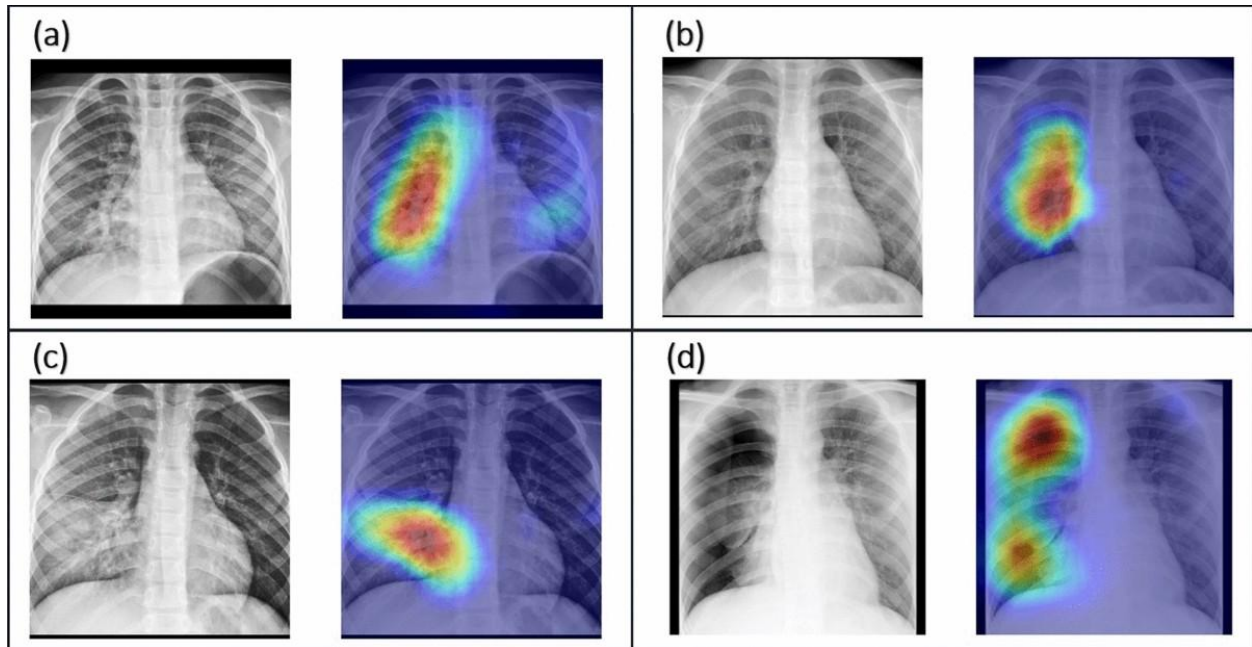
- Hai bên phổi sáng, trong, màu xám đều, không có các mảng trắng bất thường che khuất cấu trúc bên trong. Có thể nhìn rõ các mạch máu phổi tỏa ra từ rốn phổi
- Tim nằm ở vị trí trung tâm hơi lệch trái và tổng chiều ngang của tim dưới 50% kích thước của phim.
- Hai cơ hoành cong đều, cân đối. Góc sườn hoành và góc tim hoành sắc nét
- Nhìn thấy các xương sườn, xương đòn, cột sống rõ ràng. Không có khí bất thường dưới da hoặc vùng cổ.



Hình 3: X-quang phổi bình thường

2.2.2 Viêm phổi do bacteria (vi khuẩn)

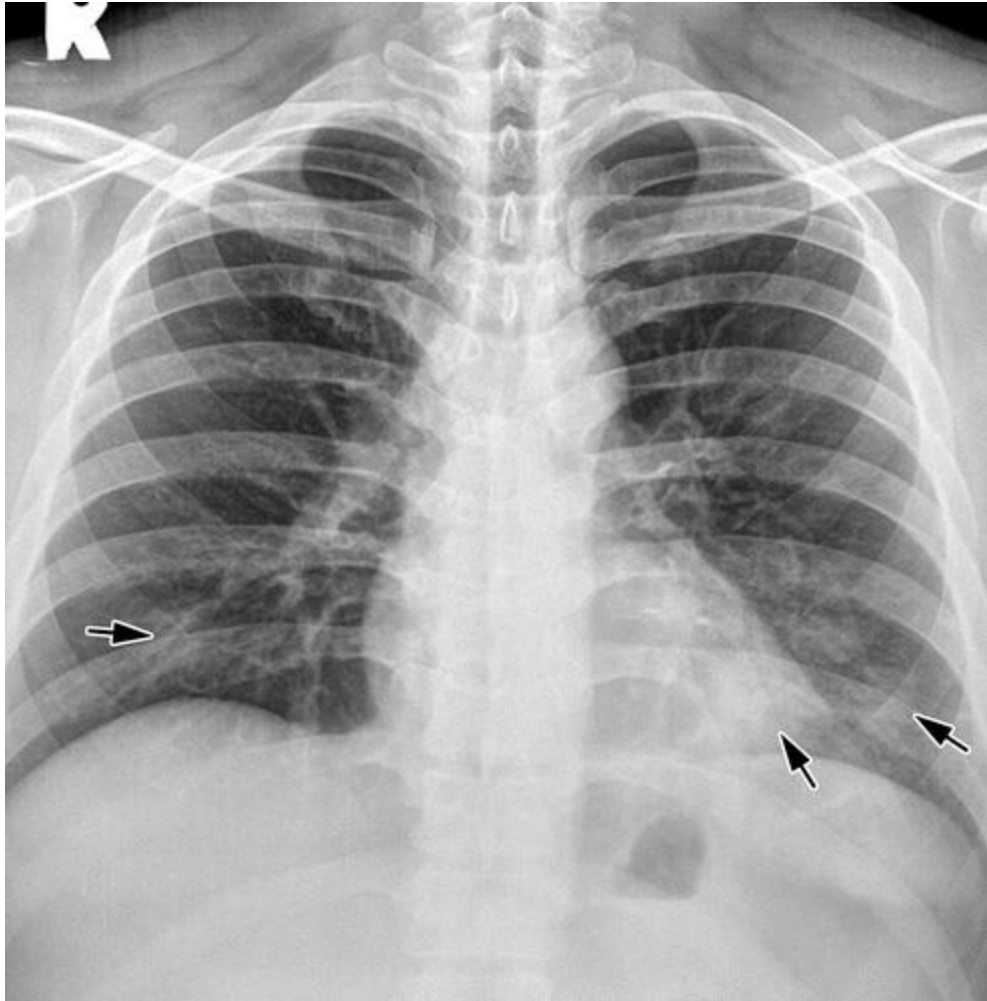
- Xuất hiện một hoặc nhiều mảng trắng đậm, khu trú ở một thùy hoặc phân thùy phổi. Các mảng trắng này có thể che lấp cấu trúc bên trong như mạch máu.
- Tim có thể bị che mờ một phần nếu tổn thương ở thùy giữa hoặc thùy dưới bên trái.
- Cơ hoành khi này sẽ không còn đều do giảm thông khí phổi dưới và bị mờ vùng đáy phổi và góc sườn hoành mất rõ nét do bệnh.
- Có các hội chứng thường gặp gây nên viêm phổi do vi khuẩn gây ra:
 - + Hội chứng phế nang: đông đặc đồng nhất hoặc không.
 - + Viêm phổi tròn: do kênh Lambert và lỗ Kohn chưa phát triển -> tổn thương khu trú tạo đám mờ giới hạn rõ.
 - + Viêm phổi do Klebsiella: thối tăng thể tích do tăng tiết dịch nhiều, đẩy cong rãnh liên thùy.
 - + Tràn dịch màng phổi.



Hình 4: X-quang phổi do vi khuẩn

2.2.3 Viêm phổi do Virus

- Xuất hiện nhiều vùng có mảng trắng hoặc lan tỏa cả hai bên phổi, không khu trú một bên như bacteria. Xuất hiện ở dạng hình kính mờ, lưới gọi là tổn thương tổ chức kẽ. Mảng trắng thường mờ hơn nên vẫn còn có thể thấy những mạch máu phía sau so với bacteria.
- Tim thường rõ không bị che mờ đi do tính chất mờ nhẹ lan tỏa. Có thể mờ khi virus đi kèm với bacteria.
- Có mờ nhẹ ở vùng đáy phổi, vẫn có thể nhìn thấy được góc sườn, tỉ lệ tràn dịch màng phổi ít hơn nhiều khi bị viêm do bacteria.
- Đặc biệt phổi bị viêm do virus có thể thấy thành phế quản dày hơn bình thường.
- Có các tổn thương thường gặp khi viêm phổi do virus:
 - + Tổn thương tổ chức kẽ: hình kính mờ, hình lưới, đường kerley
 - + Dày thành phế quản
 - + Nốt mờ: khu trú hay lan tỏa
 - + Đông đặc: khu trú hay lan tỏa, thường nhẹ hơn so với viêm do vi khuẩn



Hình 5: X-quang phổi do virus

CHƯƠNG 3: TIỀN XỬ LÝ DỮ LIỆU VÀ XÂY DỰNG MÔ HÌNH

3.1 Sử dụng ImageDataGenerator để tiền xử lý

- ImageDataGenerator là một lớp trong thư viện Keras dùng để tiền xử lý và tăng cường dữ liệu ảnh giúp sinh thêm ảnh mới bằng cách xoay, lật, phóng to, thu nhỏ ảnh,... mục đích giúp mô hình có thể học được nhiều đặc trưng hơn từ ảnh từ đó giúp nâng cao độ chính xác của mô hình và đặc biệt giảm overfitting (quá khớp) của mô hình.

```
# Chuẩn bị dữ liệu và Augmentation
# Tạo ImageDataGenerator cho tập train (bao gồm augmentation để giảm overfitting)
train_datagen = ImageDataGenerator(
    rescale=1./255,          # Chuẩn hóa giá trị pixel về [0,1]
    rotation_range=25,       # Xoay ngẫu nhiên ±25 độ
    width_shift_range=0.1,   # Dịch ngang 10%
    height_shift_range=0.1,  # Dịch dọc 10%
    zoom_range=0.2,         # Phóng to/thu nhỏ 20%
    shear_range=0.1,        # Biến dạng cắt chéo
    horizontal_flip=True     # Lật ngang ảnh
)

# Tập validation và test chỉ chuẩn hóa, không augmentation
val_test_datagen = ImageDataGenerator(rescale=1./255)
```

Hình 6: Xử lý, tăng cường dữ liệu ảnh

- Thực hiện chuẩn hóa cho tập train như trên hình với:
- + `rescale=1./255`: Chuẩn hóa giá trị pixel của ảnh từ khoảng $[0, 255]$ về khoảng $[0, 1]$. Điều này giúp mô hình hội tụ nhanh hơn.
- + Các tham số còn lại (`rotation_range`, `zoom_range`, v.v.): Đây chính là Data Augmentation. Kỹ thuật này tạo ra các phiên bản ảnh mới từ ảnh gốc bằng cách xoay, phóng to, lật, v.v. một cách ngẫu nhiên. Mục đích là để làm cho mô hình "nhìn" thấy nhiều biến thể hơn, giúp nó khái quát hóa tốt hơn và giảm thiểu overfitting (quá khớp - hiện tượng mô hình học thuộc lòng dữ liệu huấn luyện).
- + Với `rotation_range=25`, ý nghĩa là thực hiện xoay ảnh ngẫu nhiên trong ± 25 độ. Giúp mô hình học được ở nhiều góc chụp khác nhau và đặc biệt ảnh X-quang thường không hoàn toàn thẳng nên thực hiện xoay ảnh là điều cần thiết giúp cho mô hình học tốt hơn. Lưu ý nếu xoay quá lớn hoặc quá nhỏ thì có thể khiến ảnh mất đối xứng tự nhiên đặc biệt với bộ dữ liệu có cấu trúc ảnh phổi.

- + Với `width_shift_range=0.1` và `height_shift_range=0.1` thực hiện dịch chuyển theo chiều ngang dọc với 10% kích thước của ảnh gốc. Giúp mô hình có thể học được tính ổn định vị trí, mục đích là làm cho mô hình học được đa dạng vị trí của đối tượng cần học. Nếu dịch quá lớn thì đối tượng sẽ bị cắt mất từ đó học sai.
- + Với `zoom_range=0.2`, ý nghĩa là phóng to hoặc thu nhỏ ảnh trong khoảng $\pm 20\%$ ảnh gốc. Giúp mô hình nhận diện được đối tượng ở nhiều tỉ lệ khác nhau bởi vì trong bộ dữ liệu các đối tượng không chung một tỷ lệ kích thước hay khoảng cách ảnh nhất định.
- + `shear_range=0.1`, thực hiện biến dạng cắt chéo tức làm nghiêng ảnh theo trục x hoặc trục y tạo ra các ảnh có độ nghiêng nhẹ, mô phỏng sự sai lệch góc chụp trong thực tế từ đó giúp mô hình học được tốt hơn với các ảnh có độ nghiêng nhẹ trong bộ dữ liệu chest x-ray này.
- + `horizontal_flip=True`, lật ảnh ngang trái phải ngẫu nhiên tăng các mẫu đối xứng, rất phù hợp cho cấu trúc 2 bên phổi của con người trong chụp X-quang. Lưu ý ảnh y tế thường không đối xứng hoàn toàn vì thế chỉ nên dùng khi ảnh có cấu trúc hai bên giống hoặc gần giống nhau.
- + `val_test_datagen`: Tạo một công thức xử lý cho dữ liệu validation và test. Lưu ý quan trọng: Chúng ta chỉ chuẩn hóa (rescale) chứ không áp dụng augmentation cho hai tập này, vì chúng ta muốn đánh giá hiệu năng của mô hình trên dữ liệu gốc, không bị biến đổi.

3.2 Đọc và chuẩn bị hình ảnh

```
# Đường dẫn dữ liệu và tham số cơ bản
BASE_DIR = "/kaggle/input/chest-x-ray-images-normal-and-pneumonia/chest_xray"
TRAIN_DIR = os.path.join(BASE_DIR, "train")
VAL_DIR = os.path.join(BASE_DIR, "val")
TEST_DIR = os.path.join(BASE_DIR, "test")

IMG_SIZE = (224, 224) # Kích thước ảnh đầu vào
BATCH_SIZE = 32 # Số lượng ảnh xử lý mỗi batch
EPOCHS = 10 # Số vòng lặp huấn luyện
```

Hình 7: Đọc và chuẩn bị hình ảnh

- `BASE_DIR`: Thư mục gốc chứa toàn bộ dữ liệu.
- `TRAIN_DIR`, `VAL_DIR`, `TEST_DIR`: Đường dẫn đến các tập dữ liệu Huấn luyện (train), Kiểm định (validation), và Kiểm tra (test). `os.path.join` giúp nối các phần của đường dẫn lại với nhau một cách chính xác.

- **IMG_SIZE**: Kích thước mà tất cả các ảnh sẽ được thay đổi về. Mô hình yêu cầu tất cả ảnh đầu vào phải có cùng kích thước. (224, 224) là kích thước phổ biến cho các mô hình học chuyển giao.
- **BATCH_SIZE**: Số lượng ảnh được xử lý trong một lần lặp. Thay vì đưa từng ảnh vào, mô hình sẽ học từ một "lô" 32 ảnh một lúc để quá trình học ổn định và nhanh hơn.
- **EPOCHS**: Số lần mô hình sẽ "nhìn" qua toàn bộ tập dữ liệu huấn luyện. Một epoch là một vòng lặp hoàn chỉnh qua tập train.
- **Sử dụng flow_from_directory(...)**: Phương thức này sẽ đọc ảnh từ các thư mục con. Nó tự động hiểu rằng mỗi thư mục con là một lớp (ví dụ: thư mục NORMAL và PNEUMONIA). Nó sẽ tạo ra các lô (batch) dữ liệu đã được xử lý sẵn sàng để đưa vào mô hình.

```
# Tạo generator cho train, validation, test
train_gen = train_datagen.flow_from_directory(
    TRAIN_DIR,
    target_size=IMG_SIZE,
    batch_size=BATCH_SIZE,
    class_mode='binary'          # Vì bài toán nhị phân (NORMAL/PNEUMONIA)
)

val_gen = val_test_datagen.flow_from_directory(
    VAL_DIR,
    target_size=IMG_SIZE,
    batch_size=BATCH_SIZE,
    class_mode='binary'
)

test_gen = val_test_datagen.flow_from_directory(
    TEST_DIR,
    target_size=IMG_SIZE,
    batch_size=BATCH_SIZE,
    class_mode='binary',
    shuffle=False                # Không xáo trộn, để đánh giá chính xác
)
```

Hình 8: Tạo generator cho train, validation và test

- **train_gen**: Dữ liệu huấn luyện mô hình với train_datagen đã chuẩn hóa bằng ImageDataGenerator.
- **val_gen**: Dữ liệu kiểm định trong quá trình train (validation).
- **test_gen**: Dữ liệu đánh giá mô hình sau khi train xong.
- **class_mode='binary'**: Vì đây là bài toán phân loại 2 lớp (Normal/Pneumonia).

- shuffle=False cho test_gen: Không xáo trộn dữ liệu test để đảm bảo thứ tự là cố định, giúp việc đánh giá chi tiết sau này dễ dàng hơn.

3.3 Xây dựng mô hình:

- Ý tưởng xây dựng mô hình Xception với Transfer Learning vào để dự đoán ảnh phổi Chest X-Ray Images (Normal and Pneumonia).
- XceptionNet hay ngắn gọn là Xception là một mạng nơ-ron tích chập (CNN) hiện đại, rất mạnh trong việc xử lý ảnh, phân loại ảnh và trích xuất đặc trưng từ ảnh. Viết tắt của “Extreme Inception” — tức là phiên bản Inception được mở rộng đến cực độ. Được giới thiệu vào năm 2017 bởi Francois Chollet.
- Mô hình có kiến trúc gồm 36 lớp tích chập được chia thành 3 khối chính: Entry Flow, Middle Flow, Exit Flow tương ứng vai trò ở 3 giai đoạn đầu, giữa và cuối. Sử dụng Global Average Pooling thay vì Flatten để giảm overfit và Dense layer (softmax) sẽ cho đầu ra phân loại. Nên nó được sử dụng rộng rãi trong nhiều bài toán nhận dạng ảnh, như chẩn đoán ảnh X-quang nhờ khả năng học đặc trưng mạnh mẽ.
- Transfer Learning là kỹ thuật sử dụng kiến thức đã học được từ một bài toán (nguồn) để giải quyết một bài toán khác (đích) — thường có dữ liệu ít hơn hoặc hơi khác.

```
# Xây dựng mô hình XceptionNet (Transfer Learning)
# Tải base model Xception với trọng số pretrained từ ImageNet
base_model = Xception(
    input_shape=IMG_SIZE + (3,), # 3 kênh RGB
    include_top=False,           # Bỏ phần fully connected gốc của ImageNet
    weights='imagenet'           # Dùng trọng số pretrained
)
```

Hình 9: Xây dựng mô hình XceptionNet

- Sử dụng Transfer Learning: Thay vì xây dựng một mô hình từ đầu (tốn rất nhiều dữ liệu và thời gian), chúng ta lấy một mô hình đã được huấn luyện trên một bộ dữ liệu khổng lồ (ImageNet - với hàng triệu ảnh và 1000 lớp). Mô hình này đã học được cách nhận diện các đặc trưng cơ bản như cạnh, góc, kết cấu, v.v. Chúng ta sẽ "tái sử dụng" phần kiến thức này.
- base_model = Xception(...): Tải mô hình Xception.
- + include_top=False: Đây là tham số quan trọng nhất. Nó có nghĩa là chúng ta chỉ lấy phần trích xuất đặc trưng của Xception, và bỏ đi phần phân loại cuối cùng (lớp Dense dùng để phân loại 1000 lớp của ImageNet).

- + weights='imagenet': Sử dụng các trọng số đã được huấn luyện trên ImageNet.

```
# Đóng băng trọng số của base model (chưa fine-tune)
base_model.trainable = False
```

Hình 10: Đóng băng trọng số base model

- base_model.trainable = False: Đóng băng các trọng số của base_model. Chúng ta không muốn cập nhật trọng số của nó trong quá trình huấn luyện ban đầu, vì kiến thức của nó đã rất tốt rồi. Chúng ta chỉ cần huấn luyện các lớp mới mà chúng ta thêm vào.

```
# Thêm các tầng mới phía trên cho bài toán phân loại nhị phân
model = models.Sequential([
    base_model,
    layers.GlobalAveragePooling2D(), # Nén đặc trưng không gian
    layers.Dropout(0.3),             # Giảm overfitting
    layers.Dense(1, activation='sigmoid') # Đầu ra xác suất (0 hoặc 1)
])
```

Hình 11: Tạo mô hình mới bằng cách xếp chồng các lớp

- model = models.Sequential(...): Tạo một mô hình mới bằng cách xếp chồng các lớp lên nhau.
- + base_model: Lớp đầu tiên chính là toàn bộ mô hình Xception đã đóng băng.
- + GlobalAveragePooling2D(): Lớp này lấy đầu ra đặc trưng từ base_model và tính giá trị trung bình cho mỗi bản đồ đặc trưng, chuyển nó thành một vector phẳng. Cách này hiệu quả hơn việc dùng lớp Flatten vì nó giảm đáng kể số lượng tham số.
- + Dropout(0.3): Một kỹ thuật điều chuẩn (regularization) khác để chống overfitting. Trong mỗi bước huấn luyện, nó sẽ "tắt" ngẫu nhiên 30% các nơ-ron, buộc mạng lưới phải học các đặc trưng một cách dư thừa và mạnh mẽ hơn.
- + Dense(1, activation='sigmoid'): Lớp đầu ra cuối cùng.
 - o Dense(1): Có 1 nơ-ron đầu ra, vì đây là phân loại nhị phân.
 - o activation='sigmoid': Hàm kích hoạt sigmoid sẽ cho ra một giá trị trong khoảng (0, 1), có thể được hiểu là xác suất ảnh thuộc lớp "Pneumonia".

```
# Compile mô hình
model.compile(
    optimizer='adam',           # Bộ tối ưu Adam
    loss='binary_crossentropy', # Hàm mất mát cho bài toán nhị phân
    metrics=['accuracy']        # Đánh giá độ chính xác
)
```

Hình 12: Compile mô hình

- optimizer='adam': Thuật toán tối ưu hóa sẽ cập nhật trọng số của mô hình. Adam là một lựa chọn phổ biến và rất hiệu quả.
- loss='binary_crossentropy': Hàm mất mát. Nó đo lường "mức độ sai" của mô hình. binary_crossentropy là hàm mất mát tiêu chuẩn cho bài toán phân loại nhị phân.
- metrics=['accuracy']: Thước đo hiệu năng mà chúng ta muốn theo dõi. Ở đây là độ chính xác (accuracy).

```
# Đặt callback: EarlyStopping & ReduceLRonPlateau
# EarlyStopping: dừng sớm nếu val_loss không giảm
# ReduceLRonPlateau: giảm learning rate nếu mô hình không cải thiện
callbacks = [
    EarlyStopping(monitor='val_loss', patience=3, restore_best_weights=True),
    ReduceLRonPlateau(monitor='val_loss', factor=0.3, patience=2, verbose=1)
]
```

Hình 13: Earlystop và Callback nếu val_loss không giảm

- EarlyStopping: Tự động dừng huấn luyện sớm nếu một chỉ số không được cải thiện.
- + monitor='val_loss': Theo dõi hàm mất mát trên tập validation.
- + patience=3: Nếu val_loss không giảm trong 3 epoch liên tiếp, quá trình huấn luyện sẽ dừng lại.
- + restore_best_weights=True: Rất hữu ích! Nó sẽ khôi phục lại trọng số của mô hình từ epoch có val_loss tốt nhất.
- ReduceLRonPlateau: Tự động giảm tốc độ học (learning rate).
- + monitor='val_loss': Theo dõi hàm mất mát trên tập validation.
- + patience=2: Nếu val_loss không giảm trong 2 epoch, tốc độ học sẽ được giảm.
- + factor=0.3: Tốc độ học sẽ được nhân với 0.3 (giảm đi). Việc này giúp mô hình tinh chỉnh trọng số một cách cẩn thận hơn khi nó gần đạt đến điểm tối ưu.

```
# Huấn luyện mô hình
history = model.fit(
    train_gen,
    validation_data=val_gen,
    epochs=EPOCHS,
    callbacks=callbacks
)
```

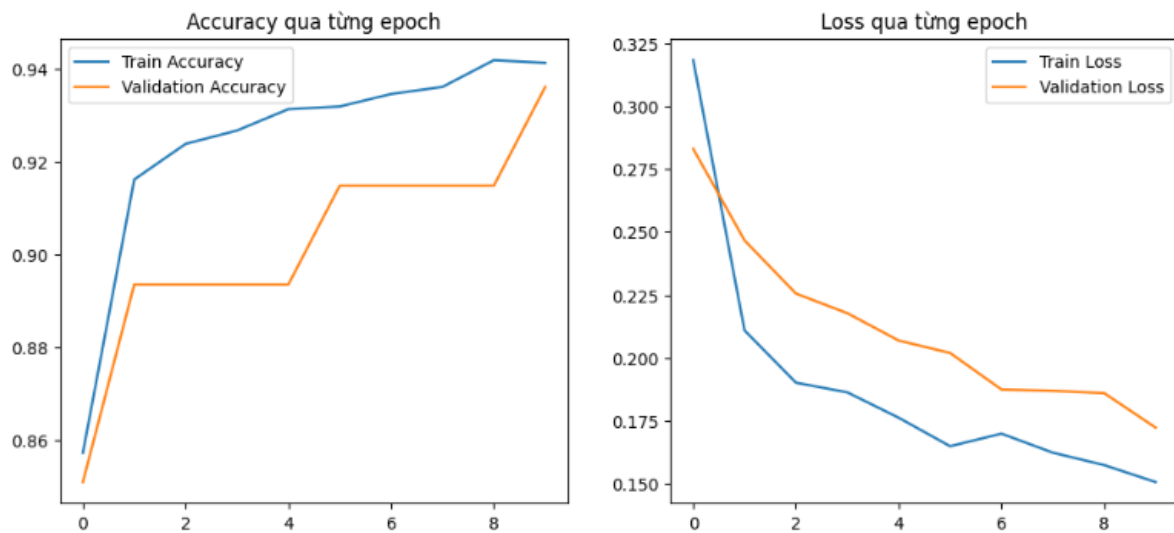
Hình 14: Huấn luyện mô hình

- `model.fit()`: Bắt đầu huấn luyện mô hình.
- Nó sẽ lấy dữ liệu từ `train_gen`, huấn luyện trong `EPOCHS` vòng lặp, sau mỗi epoch sẽ đánh giá hiệu năng trên `validation_data`, và sử dụng các `callbacks` đã định nghĩa.
- `history`: Biến này sẽ lưu lại toàn bộ lịch sử huấn luyện (`loss`, `accuracy`, `val_loss`, `val_accuracy`) qua từng epoch.

3.4 Kiểm tra kết quả Train mô hình

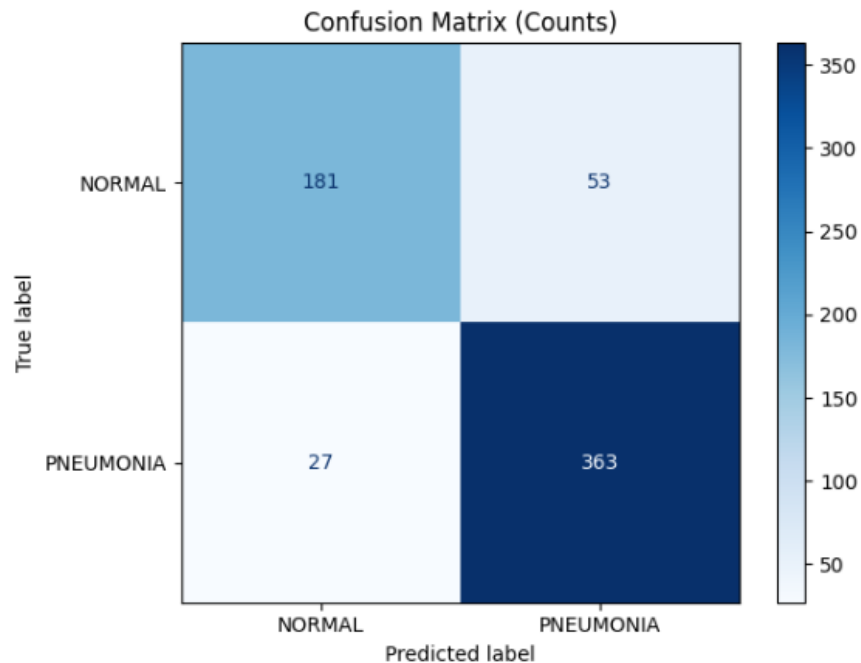
```
Epoch 8/10
163/163 ————— 91s 557ms/step - accuracy: 0.9383 - loss: 0.1601 - val_accuracy: 0.
9149 - val_loss: 0.1870 - learning_rate: 0.0010
Epoch 9/10
163/163 ————— 91s 557ms/step - accuracy: 0.9420 - loss: 0.1584 - val_accuracy: 0.
9149 - val_loss: 0.1861 - learning_rate: 0.0010
Epoch 10/10
163/163 ————— 94s 575ms/step - accuracy: 0.9459 - loss: 0.1430 - val_accuracy: 0.
9362 - val_loss: 0.1724 - learning_rate: 0.0010
20/20 ————— 11s 547ms/step - accuracy: 0.8213 - loss: 0.4142
Test Accuracy: 0.8718
Test Loss: 0.2955
```

Hình 15: Kết quả huấn luyện mô hình



Hình 16: Độ chính xác và mất mát qua từng epoch

- Nhận xét về mô hình: Train accuracy khá cao (94%) và train loss khá thấp (0,14) với 10 EPOCH và learning rate là 0.001. Nhưng khi test trên tập test thì mô hình đạt accuracy là 87,18% và loss là 0.29 thì cho thấy mô hình đang hoạt động tốt với tập train nhưng khi hoạt động trên tập test thì thấp hơn hẳn 7% nên mô hình vẫn chưa ổn khi gặp dữ liệu mới.
- Biểu đồ bên trái (Accuracy): Đường train cao hơn val mô hình học rất tốt trên train, nhưng kém hơn một chút trên val. Khoảng cách này cho thấy mô hình có overfitting nhẹ.
- Biểu đồ bên phải (Loss): Cả train loss và val loss đều giảm mô hình đang học đúng hướng. Tuy nhiên val loss vẫn cao hơn mô hình chưa tổng quát hóa tốt.



Hình 17: Ma trận nhầm lẫn giữa NORMAL và PNEUMONIA

- Nhìn vào ma trận nhầm lẫn cho thấy mô hình nhận biết được bệnh nhân bị viêm phổi tốt hơn hẳn so với người bình thường với 27 ca đoán nhầm trên 390 ảnh (6%). Nhưng cũng có thể là do sự mất cân bằng dữ liệu (ảnh PNEUMONIA nhiều hơn NORMAL) nên nó học tốt trên lớp chiếm ưu thế.

Classification Report:				
	precision	recall	f1-score	support
NORMAL	0.8702	0.7735	0.8190	234
PNEUMONIA	0.8726	0.9308	0.9007	390
accuracy			0.8718	624
macro avg	0.8714	0.8521	0.8599	624
weighted avg	0.8717	0.8718	0.8701	624

Hình 18: Báo cáo phân loại

- Thông qua Classification Report:
- + NORMAL: precision = 0.87, recall = 0.77 → mô hình bỏ sót khá nhiều ca NORMAL (recall thấp).

- + PNEUMONIA: precision = 0.87, recall = 0.93 → mô hình phát hiện bệnh nhân viêm phổi khá tốt, nhưng đôi khi báo sai người bình thường là bệnh.
- + F1-score trung bình: ~0.86 → mức khá, nhưng chưa ổn định giữa hai lớp.

3.5 Fine tuning

- Fine-tuning (tinh chỉnh mô hình) là bước huấn luyện lại một phần mô hình đã được học trước (pretrained model) để phù hợp với dữ liệu mới.
- Trong Transfer Learning, sau khi dùng mô hình có sẵn (như Xception) để trích đặc trưng chung (feature extraction), ta mở khóa một số tầng cuối (layers) của mô hình và huấn luyện lại với tốc độ học nhỏ. Điều này giúp mô hình học sâu hơn các đặc trưng riêng của tập dữ liệu mới (ví dụ ảnh X-quang), nâng cao độ chính xác, nhưng vẫn giữ được kiến thức đã học từ trước.
- Mở các lớp để fine tuning:

```
base_model.trainable = True
```

```
# Đóng băng tất cả các lớp TRỪ 20 lớp cuối cùng
# Đây là một siêu tham số, bạn có thể thử nghiệm với các giá trị khác
layers_to_freeze = len(base_model.layers) - 20
for layer in base_model.layers[:layers_to_freeze]:
    layer.trainable = False
```

```
# Huấn luyện thêm một vài epochs nữa
fine_tune_epochs = 5 # Thường chỉ cần ít epochs cho fine-tuning

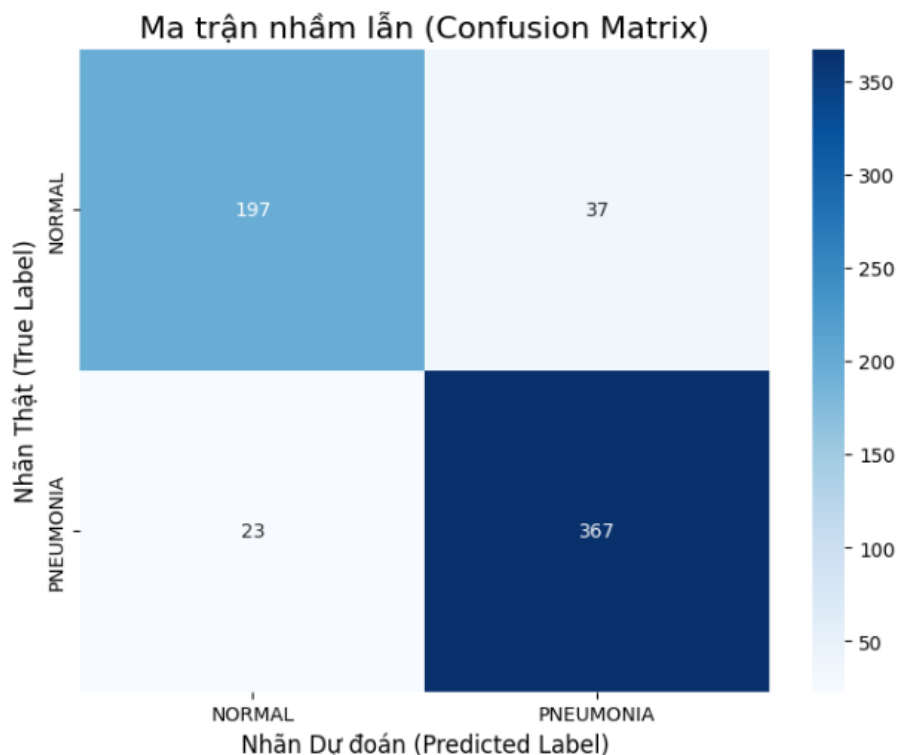
history_fine = model.fit(
    train_gen,
    validation_data=val_gen,
    epochs=fine_tune_epochs, # Chỉ chạy số epochs dành cho fine-tune
    callbacks=callbacks_ft
)
```

Hình 19: FineTuning mô hình

- Kết quả sau khi fine tuning

Epoch 3/5
163/163 ————— **89s** 548ms/step - accuracy: 0.9479 - loss: 0.1676 - val_accuracy: 0.9149 - val_loss: 0.1756 - learning_rate: 1.0000e-05
 Epoch 4/5
163/163 ————— **88s** 539ms/step - accuracy: 0.9504 - loss: 0.1437 - val_accuracy: 0.9362 - val_loss: 0.1610 - learning_rate: 1.0000e-05
 Epoch 5/5
163/163 ————— **88s** 539ms/step - accuracy: 0.9540 - loss: 0.1303 - val_accuracy: 0.9574 - val_loss: 0.1607 - learning_rate: 1.0000e-05

Hình 20: Kết quả huấn luyện sau khi FineTuning



Hình 21: Ma trận nhầm lẫn sau khi FineTuning

- Nhìn qua kết quả ma trận sau khi thực hiện fine tuning thì phần dự đoán sai của lớp PNEUMONIA giảm được 4 ca so với trước nhưng đặc biệt mô hình đã dự đoán được lớp NORMAL tốt hơn một cách rõ rệt chỉ với 37 ca dự đoán sai so với 53 khi chưa fine tuning. Điều đó cho thấy sau khi fine tuning thì mô hình đã cải thiện học được lớp thiếu số tốt hơn khi chưa fine tuning rất nhiều.

Báo cáo Phân loại (Classification Report):

	precision	recall	f1-score	support
NORMAL	0.90	0.84	0.87	234
PNEUMONIA	0.91	0.94	0.92	390
accuracy			0.90	624
macro avg	0.90	0.89	0.90	624
weighted avg	0.90	0.90	0.90	624

Hình 22: Báo cáo phân loại sau khi FineTuning

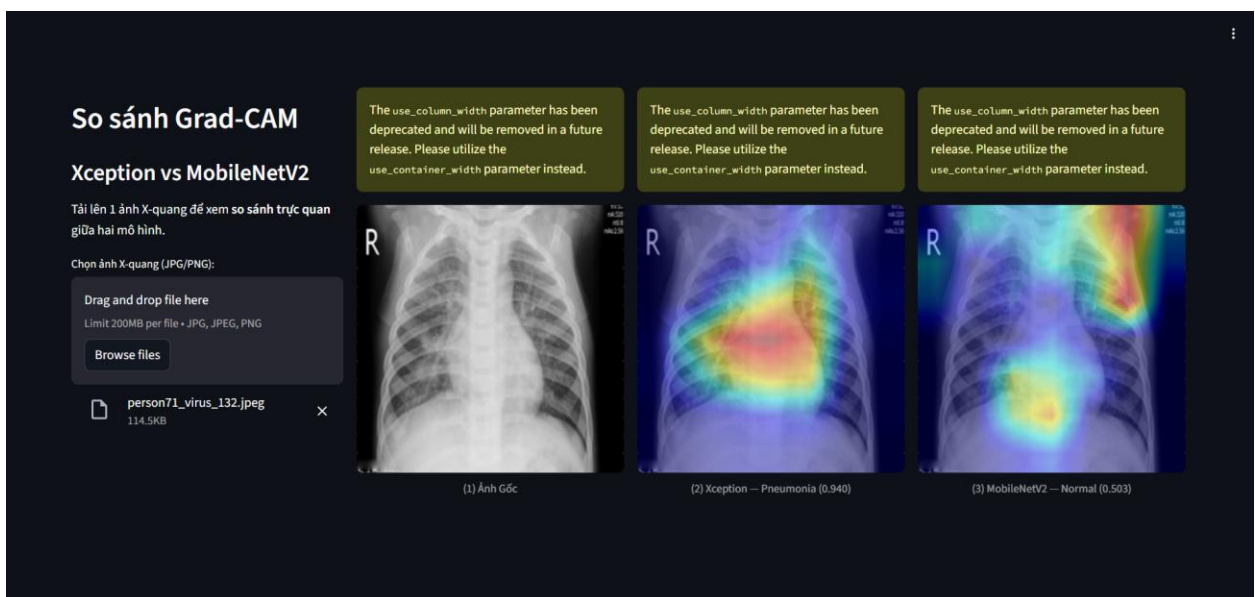
- Thông qua Classification Report sau khi fine tuning
- + NORMAL: recall tăng từ 0.77 $\rightarrow$ 0.84, f1-score từ 0.82 $\rightarrow$ 0.87
- + PNEUMONIA: recall giữ cao (0.93 $\rightarrow$ 0.94), f1-score tăng từ 0.90 $\rightarrow$ 0.92
- + Accuracy toàn hệ thống: từ 87% $\rightarrow$ 90%
- Thông qua số liệu trên cho thấy mô hình không chỉ phát hiện bệnh tốt, mà còn giảm nhầm lẫn với người bình thường, tức là độ chính xác tổng thể và độ tin cậy đều tăng. Điều này cho thấy quá trình fine-tuning đã giúp mô hình học tốt hơn đặc trưng của cả hai lớp, cải thiện cân bằng giữa hai loại ảnh.

3.6 Kết quả mô hình trên web

- Giao diện web được xây dựng bằng Streamlit để trình bày kết quả dự đoán và trực quan hóa Grad-CAM. Cho phép người sử dụng tải ảnh lên dạng jpg và png của hình ảnh X-quang cần dự đoán, sau đó hiển thị ảnh gốc và ảnh heatmap của mô hình Xception và MobileNetV2 kèm theo dự đoán có bệnh hay không bệnh kèm độ tin cậy. Mục đích sử dụng hai mô hình là để so sánh xem mô hình nào cho ra dự đoán tốt nhất trên cùng một ảnh nhanh chóng, mang lại trải nghiệm quan sát nhanh, trực quan và dễ sử dụng.
- Ứng dụng web được xây dựng kết hợp các thư viện của TensorFlow/Keras, OpenCV, NumPy và PIL, cùng với một số xử lý logic để thực hiện việc tiền xử lý ảnh, dự đoán, tạo heatmap Grad-CAM và hiển thị kết quả trực quan trên giao diện.
- Một số kết quả khi dự đoán ảnh:



Hình 23: Ảnh dự đoán và so sánh mô hình trên web 1



Hình 24: Ảnh dự đoán và so sánh mô hình trên web 2



Hình 25: Ảnh dự đoán và so sánh mô hình trên web 3



Hình 26: Ảnh dự đoán và so sánh mô hình trên web 4

CHƯƠNG 4: PHÂN TÍCH ẢNH & KẾT QUẢ

4.1 Grad-CAM

- Grad-CAM (Gradient-weighted Class Activation Mapping) là một kỹ thuật trực quan hóa giúp hiển thị vùng hình ảnh mà mô hình tập trung khi đưa ra dự đoán của nó. Sử dụng gradient dự đoán theo từng đặc trưng trong layer cuối cùng của mạng tích chập, sau đó sinh ra bản đồ nhiệt (heatmap) để biểu diễn vùng và mức độ quan trọng của vùng mà mô hình dự đoán.
- Mục đích của việc sử dụng Grad-CAM là nhằm trực quan hóa các vùng đặc trưng mà mô hình đã học được từ ảnh X-quang. Bằng cách hiển thị vùng mà mô hình tập trung khi dự đoán, Grad-CAM giúp kiểm tra xem mô hình có thật sự chú ý đến khu vực phổi – nơi chứa các dấu hiệu viêm – hay không. Từ đó, có thể đánh giá được độ tin cậy và khả năng giải thích của mô hình khi phân loại ảnh “bị viêm” và “không bị viêm”.

4.2 Code thực hiện và giải thích

- Tải lại mô hình Xception và đường dẫn ảnh để tiến hành Grad-CAM

```
# --- Tham số ---
IMG_SIZE = (224, 224)
model_path = "/kaggle/working/xception_chestxray_finetuned.h5"
image_path = "/kaggle/input/chest-x-ray-images-normal-and-pneumonia/chest_xray/test/PNEUMONIA/per
son11_virus_38.jpeg"

# --- Tải mô hình ---
model = tf.keras.models.load_model(model_path)
```

Hình 27: Tải lại mô hình Xception để tạo Grad-CAM

- Lấy mô hình con (base model)
- + Mô hình tổng model là mô hình lớn (Xception + các lớp Dense, Dropout... ở cuối).
- + Ở đây ta lấy phần Xception gốc – vì Grad-CAM hoạt động dựa trên các lớp tích chập (convolution), mà chúng chỉ nằm trong phần Xception.

```
# --- Hàm tạo Grad-CAM ---
def make_gradcam_heatmap(img_array, model, last_conv_layer_name):
    # --- Lấy base model Xception ---
    base_model = model.get_layer("xception")
```

Hình 28: Hàm tạo Grad-CAM

- Xây "mô hình trung gian"
- + Ở đây ta tạo ra một model mới (grad_model):
 - o Đầu vào là ảnh đầu vào của Xception.
 - o Đầu ra là kết quả của lớp conv cuối cùng mà ta muốn quan sát (ví dụ "block14_sepconv2_act").
- + Mục đích: chỉ để lấy ra đặc trưng của ảnh ở lớp conv cuối (nơi mô hình “nhìn thấy” các vùng quan trọng nhất).

```
# --- Xây model trung gian: từ input của Xception đến layer conv cần quan sát ---
last_conv_layer = base_model.get_layer(last_conv_layer_name)
grad_model = tf.keras.models.Model(
    inputs=base_model.input,
    outputs=last_conv_layer.output
)
```

Hình 29: Hàm xây model trung gian để lấy input và output

- Chạy ảnh qua Xception và ghi lại gradient
- GradientTape() cho phép TensorFlow ghi lại phép tính để sau đó tính gradient.
- + conv_output = grad_model(img_array) → ta chạy ảnh qua phần Xception, lấy đặc trưng đầu ra của lớp conv cuối.
- + tape.watch(conv_output) → báo với TensorFlow “hãy theo dõi biến này, lát nữa tôi cần tính đạo hàm theo nó”.
- + Sau đó ta truyền tiếp conv_output qua các lớp còn lại (GAP → Dropout → Dense) để mô phỏng dự đoán cuối cùng của model.
- + loss = preds[:, 0] → chọn xác suất của lớp ta quan tâm (ví dụ lớp Pneumonia) làm giá trị cần lấy gradient.
- Mục đích: ta muốn biết mỗi điểm trong bản đồ đặc trưng conv_output ảnh hưởng như thế nào đến kết quả dự đoán.


```
# --- Chạy forward qua Xception ---
with tf.GradientTape() as tape:
    conv_output = grad_model(img_array)
    tape.watch(conv_output)

    # Truyền tiếp qua các layer còn lại trong model Sequential
    x = model.get_layer("global_average_pooling2d")(conv_output)
    x = model.get_layer("dropout")(x)
    preds = model.get_layer("dense")(x)

    loss = preds[:, 0]
```

Hình 30: Chạy forward qua Xception

- Tính gradient
- + grads = đạo hàm của loss theo conv_output: Cho biết tầm quan trọng của từng pixel đặc trưng trong conv_output.
- + pooled_grads = trung bình gradient trên tất cả các vị trí không gian: Cho ra mức độ đóng góp trung bình của từng kênh (channel).
- Mỗi kênh đặc trưng trong lớp conv này quan trọng đến mức nào đối với dự đoán Pneumonia

```
# --- Tính gradient ---
grads = tape.gradient(loss, conv_output)
pooled_grads = tf.reduce_mean(grads, axis=(0, 1, 2))
```

Hình 31: Tính gradient

- Tạo bản đồ nhiệt
- + Lấy ra conv_output[0] vì batch chỉ có 1 ảnh.
- + Nhân từng kênh conv với trọng số pooled_grads tương ứng → làm nổi bật các vùng có đóng góp mạnh.
- + Sau đó trung bình trên tất cả kênh → ra một bản đồ 2D (heatmap).

```
conv_output = conv_output[0]
heatmap = tf.reduce_mean(tf.multiply(pooled_grads, conv_output), axis=-1)
```

Hình 32: Tạo bản đồ nhiệt

- Chuẩn hóa bản đồ
- + Cắt bỏ giá trị âm (chỉ giữ phần tích cực – vùng “có đóng góp dương” cho dự đoán).
- + Chia cho giá trị lớn nhất để chuẩn hóa về khoảng $[0,1]$.
- Trả về heatmap
- + Bản đồ nhiệt 2D này sẽ được dùng để chồng lên ảnh gốc, giúp ta nhìn thấy mô hình tập trung ở đâu (ví dụ: vùng viêm trong phổi).

```
heatmap = np.maximum(heatmap, 0)
heatmap /= np.max(heatmap) + 1e-8 # tránh chia cho 0
return heatmap
```

Hình 33: Code trả về heatmap

- Xử lý ảnh đầu vào với mục đích: Biến ảnh X-ray gốc thành dạng tensor (mảng số) mà mô hình có thể hiểu được.
- + `load_img(image_path, target_size=IMG_SIZE)`: Đọc ảnh X-ray từ đường dẫn `image_path` và resize về kích thước 224×224 pixel, đúng với kích thước mô hình Xception yêu cầu.
- + `img_to_array(img)`: Chuyển ảnh thành mảng numpy 3 chiều ($224 \times 224 \times 3$), chứa giá trị màu RGB từ 0–255.
- + `np.expand_dims(img_array, axis=0)`: Thêm 1 chiều ở đầu để mô hình hiểu là batch có 1 ảnh. Kết quả: tensor có kích thước (1, 224, 224, 3).
- + `/ 255.0`: Chuẩn hóa giá trị pixel về khoảng $[0, 1]$ (mô hình hoạt động ổn định hơn).

```
# --- Xử lý ảnh đầu vào ---
img = tf.keras.preprocessing.image.load_img(image_path, target_size=IMG_SIZE)
img_array = tf.keras.preprocessing.image.img_to_array(img)
img_array = np.expand_dims(img_array, axis=0) / 255.0
```

Hình 34: Xử lý ảnh đầu vào

- Dự đoán bằng mô hình với mục đích: Dùng mô hình đã huấn luyện (model) để dự đoán xem ảnh là bình thường hay viêm phổi.
- + `model.predict(img_array)`: Mô hình trả về xác suất dự đoán. Vì đây là mô hình nhị phân (Normal vs Pneumonia), output có dạng `[[0.87]]`, nghĩa là “87% xác suất bị viêm”.
- + `[0][0]`: Lấy giá trị số duy nhất từ mảng 2D đó (chính là xác suất Pneumonia).

- + `pred_label = "Pneumonia" if pred > 0.5 else "Normal"`: Nếu xác suất lớn hơn 0.5 thì dự đoán là viêm phổi, ngược lại là bình thường.
- + `prob = pred if pred > 0.5 else 1 - pred`: Tính lại “độ tự tin” (confidence) theo class dự đoán. Ví dụ: nếu mô hình dự đoán 0.2 (Normal) $\rightarrow$ `prob = 1 - 0.2 = 0.8` (80% chắc chắn là Normal).

```
# --- Dự đoán ---
pred = model.predict(img_array)[0][0]
pred_label = "Pneumonia" if pred > 0.5 else "Normal"
prob = pred if pred > 0.5 else 1 - pred
print(f" Dự đoán: {pred_label} ({prob:.3f})")
```

Hình 35: Dự đoán mô hình

- Chuẩn bị tạo Grad-CAM: Chỉ định layer convolution cuối cùng trong mạng Xception (nơi chứa thông tin không gian của ảnh). Tạo bản đồ nhiệt (heatmap) bằng hàm `make_gradcam_heatmap`.
- + `last_conv_layer = "block14_sepconv2_act"`: Đây là tên của layer conv cuối cùng trong XceptionNet trước khi pooling. Grad-CAM cần layer này để xem mô hình “nhìn vào đâu” trong ảnh.
- + `make_gradcam_heatmap(img_array, model, last_conv_layer)`: Gọi hàm đã định nghĩa trước đó để:
 - o Tính gradient của lớp dự đoán theo từng pixel của layer đó.
 - o Tạo heatmap biểu diễn vùng ảnh có ảnh hưởng mạnh nhất đến kết quả dự đoán.

```
# --- Tạo Grad-CAM ---
# Tên layer conv cuối cùng trong XceptionNet
last_conv_layer = "block14_sepconv2_act"

heatmap = make_gradcam_heatmap(img_array, model, last_conv_layer)
```

Hình 36: Chuẩn bị tạo Grad-CAM

- Hiển thị kết quả
- + `cv2.imread(image_path)`: Đọc ảnh X-ray từ đường dẫn (OpenCV đọc theo định dạng BGR, chứ không phải RGB).

- + `cv2.cvtColor(..., cv2.COLOR_BGR2RGB)`: Đổi lại thứ tự kênh màu thành RGB để hiển thị đúng màu (Matplotlib và PIL dùng RGB).
- + `cv2.resize(..., IMG_SIZE)`: Đưa ảnh về kích thước chuẩn 224×224 , cùng size với ảnh đã đưa vào mô hình.
- Kết quả: `img_orig` là ảnh X-ray gốc, sẵn sàng để chồng bản đồ nhiệt lên.

```
# --- Hiển thị kết quả ---
# (a) Ảnh gốc
img_orig = cv2.imread(image_path)
img_orig = cv2.cvtColor(img_orig, cv2.COLOR_BGR2RGB)
img_orig = cv2.resize(img_orig, IMG_SIZE)
```

Hình 37: Kết quả

- Resize heatmap cho khớp ảnh:
- + `cv2.resize(heatmap, (IMG_SIZE[0], IMG_SIZE[1]))`: Bản đồ nhiệt (heatmap) tạo từ lớp conv có kích thước nhỏ (ví dụ 14×14), nên cần phóng to lên bằng kích thước ảnh gốc (224×224) để có thể chồng chính xác.
- + `np.uint8(255 * heatmap_resized)`: Chuyển heatmap từ giá trị 0–1 sang 0–255 (chuẩn màu ảnh 8-bit).
- + `cv2.applyColorMap(..., cv2.COLORMAP_JET)` Áp màu nhiệt (đỏ-vàng-xanh) lên bản đồ để dễ nhìn:
 - o vùng xanh → nơi mô hình chú ý mạnh nhất.
 - o vùng đỏ → ít quan trọng.
- Kết quả: `heatmap_color` là ảnh màu thể hiện vùng chú ý của mô hình.

```
# (b) Resize heatmap cho khớp
heatmap_resized = cv2.resize(heatmap, (IMG_SIZE[0], IMG_SIZE[1]))
heatmap_resized = np.uint8(255 * heatmap_resized)
heatmap_color = cv2.applyColorMap(heatmap_resized, cv2.COLORMAP_JET)
```

Hình 38: Resize heatmap

- Chồng ảnh:
- + `cv2.addWeighted()` trộn 2 ảnh lại:
 - o `img_orig` (ảnh X-ray thật) chiếm 60% độ sáng.
 - o `heatmap_color` (vùng mô hình tập trung) chiếm 40%.

- + Kết quả: ảnh mới vừa nhìn thấy phổi rõ ràng, vừa thấy vùng màu đỏ mà mô hình chú ý.
- Kết quả: `superimposed_img` = ảnh X-ray có vùng “nóng” hiển thị, giúp ta đánh giá xem mô hình có nhìn đúng vùng viêm không.

```
# (c) Chồng ảnh
superimposed_img = cv2.addWeighted(img_orig, 0.6, heatmap_color, 0.4, 0)
```

Hình 39: Chồng ảnh

- Sau đó in ra 3 ảnh với ảnh gốc, biểu đồ heatmap và ảnh chồng hiển thị vùng mô hình dự đoán trên ảnh gốc.

```
# --- Vẽ biểu đồ ---
plt.figure(figsize=(14,5))
plt.subplot(1,3,1)
plt.imshow(img_orig)
plt.title("test x-ray input")
plt.axis("off")

plt.subplot(1,3,2)
plt.imshow(heatmap, cmap='viridis')
plt.title("class activation map (XceptionNet)")
plt.axis("off")

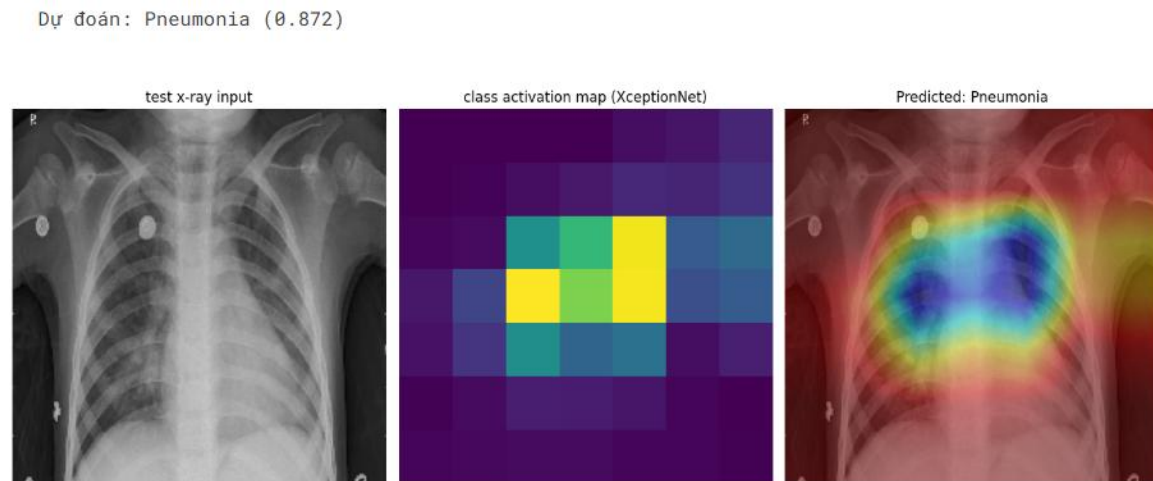
plt.subplot(1,3,3)
plt.imshow(superimposed_img)
plt.title(f"Predicted: {pred_label}")
plt.axis("off")

plt.tight_layout()
plt.show()
```

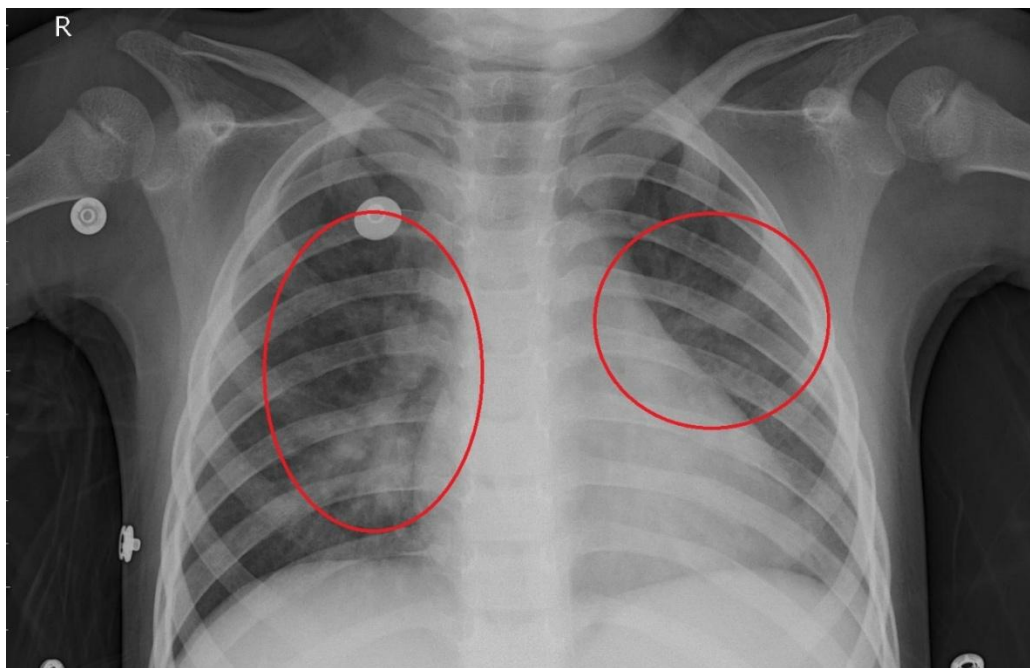
Hình 40: In ra ảnh gốc, heatmap và ảnh chồng hiển thị vùng mô hình dự đoán

4.3 Kết quả và phân tích với ảnh gốc

4.3.1 Ảnh 1



Hình 41: Ảnh sau khi mô hình tạo Grad-CAM 1



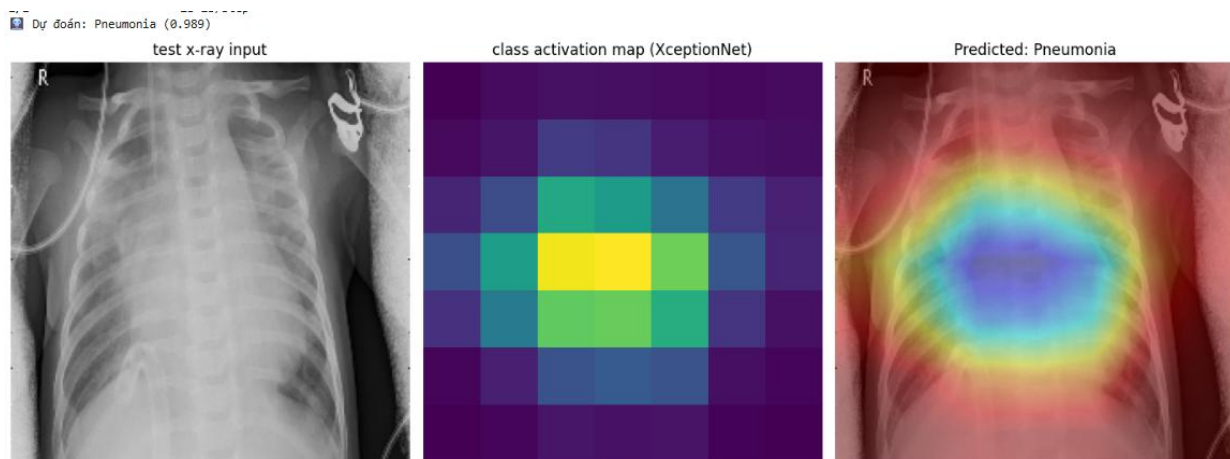
Hình 42: Ảnh gốc đã phân tích 1

- Phân tích:
- + Ở ảnh gốc X-quang trên ta thấy hai bên phổi có nhiều vùng mờ trắng và phân bố khá đều. Tuy nhiên nhìn bằng mắt thường vẫn quan sát rõ được tim có nghĩa là phổi không bị che

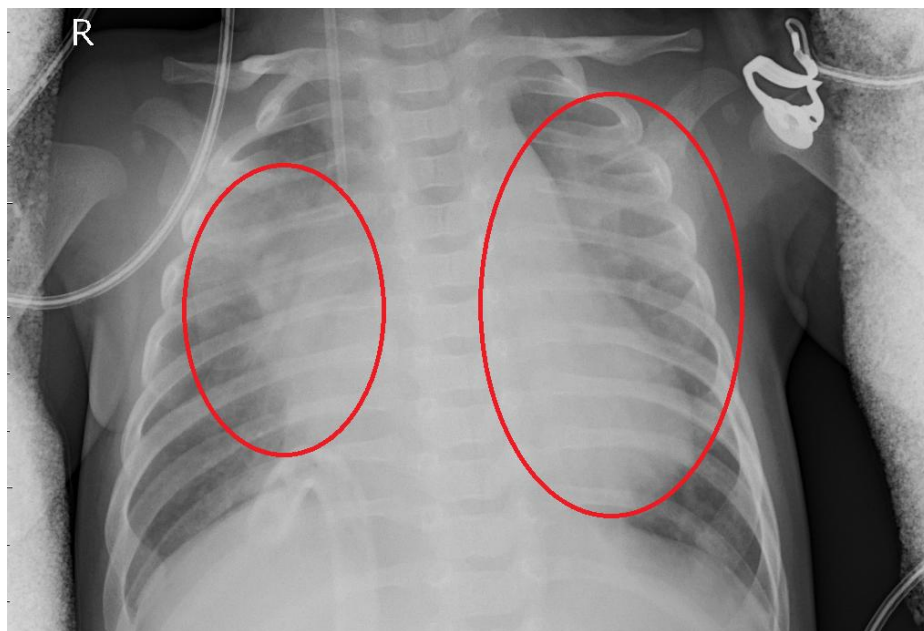
mờ hoàn toàn. Dựa vào đặc điểm trên và cơ sở lý thuyết đã nêu có thể nhận định đây là dạng viêm phổi do virus.

- + Ảnh heatmap từ kết quả của mô hình cho thấy vùng nó dự đoán nằm đúng ở giữa hai phổi, trùng với điểm đã phân tích sự bất thường của ảnh. Điều này cho thấy mô hình có vẻ là đã chú ý đúng vào vị trí có dấu hiệu bệnh (bất thường)
- + Mô hình dự đoán là PNEUMONIA với độ tin cậy 0.843, phù hợp với quan sát thực tế và đường dẫn ảnh đã được đưa vào dự đoán.

4.3.2 Ảnh 2



Hình 43: Ảnh sau khi mô hình tạo Grad-CAM 2



Hình 44: Ảnh gốc đã phân tích 2

- Phân tích:
- + Ở ảnh gốc X-quang cho ta thấy phổi của bệnh nhi này đã bị vùng trắng mờ che phủ hoàn toàn tim. Dựa vào đặc điểm trên và cơ sở lý thuyết có thể nhận định đây là dạng viêm phổi do vi khuẩn gây nên
- + Ảnh heatmap từ kết quả của mô hình cho thấy vùng nó dự đoán nằm trung tâm ở hai phổi, trùng với điểm đã phân tích sự bất thường của ảnh. Điều này cho thấy mô hình có vẻ là đã chú ý đúng vào vị trí có dấu hiệu bệnh (bất thường).
- + Mô hình dự đoán là PNEUMONIA với độ tin cậy 0.989, phù hợp với quan sát thực tế và đường dẫn ảnh đã được đưa vào dự đoán.

4.3.3 Ảnh 3



Hình 45: Ảnh sau khi mô hình tạo Grad-CAM 3



Hình 46: Ảnh gốc đã phân tích 3

- Phân tích:
- + Đây là ảnh bệnh nhi không mắc bệnh viêm phổi bởi vì có thể nhìn rõ được cấu trúc của tim, các mạch máu màu xám có thể thấy được qua ảnh. Cơ hoành đều mà rõ ràng ở 2 góc.
- + Các vùng mờ màu trắng ở đây là gì: Chụp X-quang ở trẻ em nó rất khác so với người lớn, nhất là trẻ từ một đến năm tuổi, không thể yêu cầu các bé hít sâu cho đúng quy trình chụp X-quang như người lớn được, nên phổi có thể không nở hoàn toàn, làm xuất hiện vùng mờ nhẹ do chồng mô mềm, xương sườn, hoặc cơ quan trung thất. Ngoài ra, do thành ngực mỏng và thể tích phổi nhỏ, hình ảnh mạch máu và tim tương đối chiếm diện tích lớn hơn, khiến ảnh nhìn “trắng” hơn người lớn dù vẫn bình thường.
- + Ảnh heatmap từ kết quả của mô hình cho thấy vùng nó dự đoán trên góc phải của phổi bên phải, và một chút ở góc dưới trái bên phổi trái trên cơ hoành bên trái. Tuy nhiên đây là ảnh phổi bình thường nên vùng hiển thị được mô hình cho là các đặc trưng để nhận dạng
- + Mô hình dự đoán là NORMAL với độ tin cậy 0.655, phù hợp với quan sát thực tế và đường dẫn ảnh đã được đưa vào dự đoán nhưng độ tin cậy không được cao. Có lẽ là vùng trắng mờ đó khiến mô hình mất tự tin khi dự đoán.

CHƯƠNG 5: KẾT LUẬN

5.1 Tổng kết nội dung đã thực hiện

- Tìm hiểu về cơ sở lý thuyết của ảnh phổi để phân biệt ảnh phổi nào là ảnh phổi bình thường, ảnh viêm phổi do bacteria hoặc viêm phổi, từ đó có thể biết được mô hình dự đoán như thế nào.
- Tiền xử lý dữ liệu sử dụng ImageDataGenerator để thực hiện tăng cường, chuẩn hóa ảnh. Giúp mô hình có thể học được nhiều đặc điểm từ ảnh phổi X-quang hơn, nhằm cải thiện độ chính xác, hiệu quả, độ tin cậy nhất là trong bộ dữ liệu Chest X-ray nói riêng và chẩn đoán y tế nói chung.
- Xây dựng, thử nghiệm và đánh giá nhiều mô hình khác nhau để xem được mô hình nào tốt nhất trong bộ dữ liệu này nói riêng và các bộ dữ liệu khác nữa.
- Lựa chọn mô hình kết hợp với Transfer Learning (học chuyển giao) và fine tuning cho mô hình có hiệu suất cao nhất để nâng cao độ chính xác và khả năng tổng quát hóa.
- Trực quan hóa đặc trưng mà mô hình học được bằng Grad-CAM để biết được vùng nào được mô hình tập trung đoán và hiểu hơn về cách mô hình đưa ra quyết định. Kết quả trên sau đó được so sánh với cơ sở lý thuyết nhằm đánh giá mức độ hợp lý và độ tin cậy trong quá trình nhận diện.
- Website dự đoán và so sánh 2 mô hình là MobileNetV2 và Xception, kết quả hiển thị trực quan hóa bằng Grad-CAM, có đưa ra dự đoán và độ tin cậy của 2 mô hình.

5.2 Khó khăn, thách thức và cách giải quyết

- Trong quá trình nghiên cứu, nhóm đã gặp không ít khó khăn để hiểu được ảnh phổi X-quang vì cấu trúc phức tạp của ảnh X-quang. Kèm theo việc muốn phân loại được ảnh trên bằng mắt thường thì đòi hỏi người đọc phải có kiến thức chuyên môn sâu và rộng để có thể nhận biết được các dấu hiệu bệnh lý từ đó đưa ra được kết quả chính xác là thách thức rất lớn đối với người chưa từng học và đọc ảnh phổi trước đây.
- Ngoài ra bộ dữ liệu có hơn 5800 ảnh vẫn chưa đủ lớn so với các mô hình học sâu vốn cần nhiều dữ liệu để tránh bị overfitting (quá khớp). Bộ dữ liệu X-quang trên còn không cân bằng giữa các lớp NORMAL và PNEUMONIA dẫn đến mô hình có xu hướng thiên về lớp

có nhiều mẫu hơn. Việc thiếu các chuyên gia chuyên về lĩnh vực này cũng là một thử thách lớn đối với nhóm vì nhóm chỉ có thể tham khảo các kiến thức từ các tài liệu, nền tảng học thuật và nghiên cứu công khai trên Internet. Điều này làm cho việc đối chiếu giữa kết quả mô hình và cơ sở y khoa thực tế trở nên hạn chế.

- Để giải quyết các khó khăn và thách thức trên mặc dù không triệt để nhưng nhóm cũng đã có sử dụng kỹ thuật tăng cường dữ liệu (Data Augmentation) và áp dụng Transfer Learning cho mô hình nhằm cải thiện được hiệu suất của mô hình đối với bộ dữ liệu trên. Nhóm cũng đã tham khảo nhiều tài liệu y học về ảnh phổi X-quang được công bố công khai đồng thời so sánh kết quả của mô hình với cơ sở lý thuyết thu thập được nhằm đánh giá mức độ hợp lý và độ tin cậy của mô hình.

HƯỚNG PHÁT TRIỂN

- Dựa vào nội dung đã thực hiện và các hạn chế khi thực hiện đề án trên thì nhóm có đưa ra hướng phát triển như sau:
- + Thu thập thêm nhiều ảnh X-quang từ nhiều nguồn uy tín khác nhau, với độ phân giải và góc chụp khác nhau để đa dạng hóa dữ liệu giúp tăng tính tổng quát và độ tin cậy cho mô hình
- + Có thể phân loại chi tiết hơn cụ thể là nhiều lớp hơn do mô hình hiện tại tập trung chủ yếu là NORMAL và PNEUMONIA mặc dù đã có chia ra 3 lớp là NORMAL, BACTERIA và VIRUS nhưng kết quả vẫn chưa tốt cho nên nó cũng là một hướng phát triển tốt cho dự án sau này
- + Có thể áp dụng nhiều mô hình mạnh mẽ, tiên tiến hơn như Vision Transformer, ConvNeXt kết hợp với kỹ thuật ensemble learning để cải thiện hiệu suất dự đoán cũng như đa dạng trong việc hiểu và phân loại các mô hình nào phù hợp với bộ dữ liệu y tế này.
- + Phát triển giao diện website hoặc ứng dụng desktop có khả năng nhận ảnh và dự đoán tức thì từ thiết bị chụp, hỗ trợ giúp các chuyên gia tham khảo, huấn luyện và đánh giá nhanh hình ảnh.
- + Sử dụng các kỹ thuật giải thích mô hình khác ngoài Grad-CAM như LIMP hoặc các kỹ thuật liên quan để hiểu rõ hơn cơ chế đưa ra quyết định của các mô hình sử dụng, tăng tính minh bạch và độ tin cậy khi được sử dụng trong y học.
- + Có thể public mô hình lên các diễn đàn, các nền tảng hỗ trợ như AWS hoặc Hugging Face Spaces và nhiều nền tảng khác giúp người dùng có thể truy cập, thử nghiệm và đánh giá trực tuyến.

TÀI LIỆU THAM KHẢO

- [1] Nguyễn Quang Trọng. *Đọc phim X-quang ngực [Video]*. Kênh Cận Lâm Sàng. YouTube.
<https://www.youtube.com/watch?v=JEGwXs8KJKU&t>
- [2] “Imaging of Pulmonary Infection,” NCBI Bookshelf. [Online]. Available:
<https://www.ncbi.nlm.nih.gov/books/NBK553877/>
- [3] Radiology Assistant, “Chest X-Ray: Basic Interpretation,” [Online]. Available:
<https://radiologyassistant.nl/chest/chest-x-ray/basic-interpretation>
- [4] J. P. Cohen, P. Bertin, and V. Frappier, “Chester: A Web Delivered Locally Computed Chest X-ray Disease Prediction System,” *arXiv preprint arXiv:1901.11210*, 2019. [Online]. Available:
<https://mlmed.org/tools/xray/>